

Graph

BCC

```
#include <vector>
#include <algorithm>

using namespace std;

struct BlockCutTree {
    int n, timer, bcc_cnt;
    vector<vector<int>> adj, tree;
    vector<int> dfn, low, stk;

    BlockCutTree(int _n) {
        n = _n;
        timer = bcc_cnt = 0;
        adj.resize(n + 1);
        tree.resize(n + 1);
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int u, int p = 0) {
        dfn[u] = low[u] = ++timer;
        stk.push_back(u);

        for (int v : adj[u]) {
            if (v == p) continue;

            if (dfn[v]) {
                low[u] = min(low[u], dfn[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);

                if (low[v] >= dfn[u]) {
                    bcc_cnt++;
                    int bcc_node = n + bcc_cnt;

```

```

        if (bcc_node >= tree.size()) {
            tree.resize(bcc_node + 1);
        }

        while (true) { ``
            int curr = stk.back();
            stk.pop_back();
            tree[bcc_node].push_back(curr);
            tree[curr].push_back(bcc_node);
            if (curr == v) break;
        }
        tree[bcc_node].push_back(u);
        tree[u].push_back(bcc_node);
    }
}

}

}

void build() {
    for (int i = 1; i <= n; i++) {
        if (!dfn[i]) {
            dfs(i);
        }
    }
}

};

```

Euler Path and circuit

Definition

An Eulerian path is a path in a graph that visits every edge exactly once. An Eulerian circuit is an Eulerian path that starts and ends at the same vertex.

Existence Conditions

For Undirected Graphs

- A graph has an Eulerian path if and only if:
- All edges belong to the same connected component
- Either all vertices have even degree, or exactly two vertices have odd degree (the start and end vertices)
- A graph has an Eulerian circuit if and only if all vertices have even degree

- Undirected :

For Directed Graphs

- A graph has an Eulerian path if and only if:
- All edges belong to the same connected component
- Either all vertices have equal in-degree and out-degree, or one vertex has out-degree = in-degree + 1 (start vertex), another has in-degree = out-degree + 1 (end vertex), and all others have equal in-degree and out-degree
- A graph has an Eulerian circuit if and only if all vertices have equal in-degree and out-degree

```
// 1-based
struct Eulerian {
    int n;
    bool directed;
    int edge_cnt = 0;
    vector<vector<pair<int,int>>> g; // (to, edgeID)
    vector<int> in, out;           // degree counters
    vector<int> used;              // mark edges used in DFS
    vector<int> ptr;               // iterator per node
    vector<int> path;              // final result

    Eulerian(int _n, bool _directed = false) {
        n = _n;
        directed = _directed;
        g.assign(n + 1, {});
        in.assign(n + 1, 0);
        out.assign(n + 1, 0);
    }

    void addEdge(int u, int v) {
        ++edge_cnt;
        g[u].push_back({v, edge_cnt});
        out[u]++, in[v]++;
        if (!directed) {
            g[v].push_back({u, edge_cnt});
            out[v]++, in[u]++;
        }
    }

    void dfs(int u) {
        while (ptr[u] < (int)g[u].size()) {
            auto [v, id] = g[u][ptr[u]++];
```

```

        if (used[id]) continue;
        used[id] = 1;
        dfs(v);
    }
    path.push_back(u);
}

bool solve() {
    path.clear();
    ptr.assign(n + 1, 0);
    used.assign(edge_cnt + 1, 0);

    int start = 0, end = 0, root = 0;

    if (!directed) {
        // undirected: at most 2 odd-degree vertices
        for (int i = 1; i <= n; i++) {
            if (out[i] & 1) {
                if (!start) start = i;
                else if (!end) end = i;
                else return false;
            }
        }
        if (!start) { // all even
            for (int i = 1; i <= n; i++)
                if (out[i]) { start = i; break; }
        }
        if (!start) return true; // empty graph
        root = start;
    }
    else {
        // directed: in/out degree constraints
        int cnt_start = 0, cnt_end = 0;
        for (int i = 1; i <= n; i++) {
            if (out[i] - in[i] == 1) { cnt_start++; start = i; }
            else if (in[i] - out[i] == 1) cnt_end++;
            else if (abs(in[i] - out[i]) > 1) return false;
        }
        if ((cnt_start != 1 || cnt_end != 1) && cnt_start != 0) return
false;

        if (!start) {
            for (int i = 1; i <= n; i++)
                if (out[i]) { start = i; break; }
        }
        if (!start) return true; // empty graph
        root = start;
    }
}

```

```

    }

    dfs(root);

    // connectivity check
    if ((int)path.size() != edge_cnt + 1) return false;

    reverse(path.begin(), path.end());
    return true;
}

bool isCircuit() {
    if (!solve()) return false; // if no Eulerian path, can't be circuit
    if (!directed) {
        for (int i = 1; i <= n; i++)
            if (out[i] & 1) return false;
        return true;
    }
    for (int i = 1; i <= n; i++)
        if (in[i] != out[i]) return false;
    return true;
}

vector<int> getPath() { return path; }
};

```

Floyd and bellman Ford

```

int n, m, q;
cin >> n >> m >> q;
const int INF = 1e15;
vector<vector<int>> d(n + 1, vector<int>(n + 1, INF));
// the self edge is zero
for (int i = 1; i <= n; i++) d[i][i] = 0;
while (m--) {
    int u, v, c;
    cin >> u >> v >> c;
    d[u][v] = min(d[u][v], c);
    d[v][u] = min(d[v][u], c);
}
// floyd

for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)

```

```

        for (int j = 1; j <= n; j++)
            if (d[i][k] < INF and d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    while (q--) {
        int a, b;
        cin >> a >> b;
        int ans = min(d[a][b], d[b][a]);
        if (ans == INF) cout << "-1\n";
        else cout << ans << endl;
    }

```

```

struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (Edge e : edges)
            if (d[e.a] < INF)
                d[e.b] = min(d[e.b], d[e.a] + e.cost);
    // display d, for example, on the screen
}

```

Hungarian

```

//O(n^3) or (n^2 * e)

//Hungarian Algorithm for min cost bipartite matching, works also with
negative costs
//All the nodes on the left should have an edge with all the nodes on the
right (complete graph)
//a[1][1] represents the cost of the edge between node 1 in the left and node
1 in the right
//One based and every row should be matched with exactly one column

int hung()
{

```

```

    vector<int> u (n+1), v (m+1), p (m+1), way (m+1); //n is the nodes on the
left (rows), m in the nodes on right (columns)
    for (int i=1; i<=n; ++i) {
        p[0] = i;
        int j0 = 0;
        vector<int> minv (m+1, INF);
        vector<char> used (m+1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], delta = INF, j1;
            for (int j=1; j<=m; ++j)
                if (!used[j]) {
                    int cur = a[i0][j]-u[i0]-v[j];
                    if (cur < minv[j])
                        minv[j] = cur, way[j] = j0;
                    if (minv[j] < delta)
                        delta = minv[j], j1 = j;
                }
            for (int j=0; j<=m; ++j)
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    minv[j] -= delta;
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }

    vector<int> ans (n+1);
    for (int j=1; j<=m; ++j)
        ans[p[j]] = j; //ans[p[j]] represents that j-th column is matched with
p[j]-th row

    int cost = -v[0]; //min cost
    return cost;
}

```

Example:

```

n = 2;
m = 3;
for(int i=1; i<=n; i++)

```

```

{
    for(int j=1; j<=m; j++)
    {
        int x;
        cin >> x;
        a[i][j] = x;
    }
}
cout << hung() << '\n';

```

LCA

```

const int N = 1e5 + 10, LG = 22;
int n;
int parent[N][LG];
int lvl[N];
vector<int> adj[N];

void build(int node, int par, int d) {
    parent[node][0] = par;
    lvl[node] = d;
    for (int i = 1; i < LG; i++) {
        parent[node][i] = parent[parent[node][i - 1]][i - 1];
    }
    for (auto &child: adj[node]) {
        if (child == par)continue;
        build(child, node, d + 1);
    }
}

int kth(int node, int k) {
    for (int i = LG - 1; i >= 0; i--) {
        if (k >> i & 1) {
            node = parent[node][i];
        }
    }
    return node;
}

int LCA(int u, int v) {
    if (lvl[u] < lvl[v])swap(u, v);
    u = kth(u, lvl[u] - lvl[v]);
    if (u == v)return u;
    for (int i = LG - 1; i >= 0; i--) {
        if (parent[u][i] != parent[v][i])u = parent[u][i], v = parent[v][i];
    }
    return parent[u][0];
}

```



```

[i];
    }
    return parent[u][0];
}

int dist(int u, int v) {
    int lc = LCA(u, v);
    return lvl[u] + lvl[v] - 2 * lvl[lc];
}

```

Matching

1. Maximum Bipartite Matching

- A matching is a set of edges such that no two share a vertex.
- Kuhn's algorithm (DFS based augmenting paths) finds maximum matching.
- Complexity: $O(V * E)$ where $V = n$ (left part), $E = \text{edges}$.

2. Minimum Vertex Cover

- A vertex cover = set of vertices touching all edges.
- Minimum Vertex Cover = smallest such set.
- König's Theorem (for bipartite graphs): $\text{size(MVC)} = \text{size(Maximum Matching)}$.
- We can construct the set using BFS/DFS after Kuhn.

```

struct Kuhn {
    int n, m; // left size (n), right size (m)
    vector<vector<int>> g; // adjacency list from left → right
    vector<int> mt; // match for right side
    vector<int> used; // visited marker for left
    int timer = 1;

    Kuhn(int n, int m) : n(n), m(m) {
        g.assign(n, {});
        mt.assign(m, -1);
        used.assign(n, 0);
    }

    void addEdge(int u, int v) {
        // u ∈ [0..n-1], v ∈ [0..m-1]
        g[u].push_back(v);
    }

    bool dfs(int v) {

```

```

    if (used[v] == timer) return false;
    used[v] = timer;
    for (int to : g[v]) {
        if (mt[to] == -1 || dfs(mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int maxMatching() {
    int matchSize = 0;
    for (int v = 0; v < n; v++) {
        timer++;
        if (dfs(v)) matchSize++;
    }
    return matchSize;
}

vector<pair<int,int>> getMatching() {
    vector<pair<int,int>> res;
    for (int r = 0; r < m; r++) {
        if (mt[r] != -1) res.push_back({mt[r], r});
    }
    return res;
}

// -----
// Construct Minimum Vertex Cover using König's theorem
// Returns: {leftCover, rightCover}
// Complexity: O(V + E)
// -----
pair<vector<int>, vector<int>> minVertexCover() {
    maxMatching(); // ensure we have maximum matching

    vector<int> visL(n, 0), visR(m, 0);
    queue<int> q;

    // Start BFS from unmatched vertices in left part
    vector<int> matchL(n, -1);
    for (int r = 0; r < m; r++)
        if (mt[r] != -1) matchL[mt[r]] = r;

    for (int u = 0; u < n; u++) {
        if (matchL[u] == -1) {

```

```

        q.push(u);
        visL[u] = 1;
    }
}

// BFS alternating between free/matched edges
while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) {
        if (!visR[v] && matchL[u] != v) {
            visR[v] = 1;
            if (mt[v] != -1 && !visL[mt[v]]) {
                visL[mt[v]] = 1;
                q.push(mt[v]);
            }
        }
    }
}

// Left cover = all not visited
// Right cover = all visited
vector<int> coverL, coverR;
for (int u = 0; u < n; u++)
    if (!visL[u]) coverL.push_back(u);
for (int v = 0; v < m; v++)
    if (visR[v]) coverR.push_back(v);

return {coverL, coverR};
}
};

```

Hopcroft

```

#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct HopcroftKarp {
    static const int inf = 1e9;
    int n;
    vector<int> l, r, d;
    vector<vector<int>> g;
    HopcroftKarp(int _n, int _m) {

```

```

n = _n;
int p = _n + _m + 1;
g.resize(p);
l.resize(p, 0);
r.resize(p, 0);
d.resize(p, 0);
}
void add_edge(int u, int v) {
    g[u].push_back(v + n); //right id is increased by n, so is l[u]
}
bool bfs() {
    queue<int> q;
    for (int u = 1; u <= n; u++) {
        if (!l[u]) d[u] = 0, q.push(u);
        else d[u] = inf;
    }
    d[0] = inf;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (auto v : g[u]) {
            if (d[r[v]] == inf) {
                d[r[v]] = d[u] + 1;
                q.push(r[v]);
            }
        }
    }
    return d[0] != inf;
}
bool dfs(int u) {
    if (!u) return true;
    for (auto v : g[u]) {
        if (d[r[v]] == d[u] + 1 && dfs(r[v])) {
            l[u] = v;
            r[v] = u;
            return true;
        }
    }
    d[u] = inf;
    return false;
}
int maximum_matching() {
    int ans = 0;
    while (bfs()) {
        for (int u = 1; u <= n; u++) if (!l[u] && dfs(u)) ans++;
    }
}

```

```

        return ans;
    }
};

int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int n, m, q;
    cin >> n >> m >> q;
    HopcroftKarp M(n, m);
    while (q--) {
        int u, v;
        cin >> u >> v;
        M.add_edge(u, v);
    }
    cout << M.maximum_matching() << '\n';
    return 0;
}

```

Max Flow

```

struct FlowEdge {
    int v, u;
    long long cap, flow = 0;

    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;

    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, int cap = 1) {
        edges.emplace_back(v, u, cap);
        adj[v].push_back(m++);
    }
}

```

```

        edges.emplace_back(u, v, 0);
        adj[u].push_back(m++);
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id: adj[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int &cid = ptr[v]; cid < (int) adj[v].size(); cid++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap -
edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    void dfs(int nd, vector<vector<int>> &ret, vector<int> &cur) {
        cur.push_back(nd);
        if (nd == t) {
            ret.push_back(cur);
            cur.pop_back();
        }
    }

```

```

        return;
    }

    for (int &cid = ptr[nd]; cid < (int) adj[nd].size();) {
        int id = adj[nd][cid];
        cid++;
        if ((id & 1) || (edges[id].flow == 0))continue;
        int u = edges[id].u;
        dfs(u, ret, cur);
        if (nd != s) {
            break;
        }
    }

    if (cur.empty()) {
        cout << -1;
        return;
    }
    cur.pop_back();
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}

vector<vector<int>> paths() {
    vector<vector<int>> ret;
    vector<int> cur;
    fill(ptr.begin(), ptr.end(), 0);
    dfs(s, ret, cur);
    return ret;
}

};

```

MCMF

```
struct Edge {
    int to;
    int cost;
    int cap, flow, backEdge;
};

struct MCMF {
    const int inf = 1000000010;
    int n;
    vector<vector<Edge>> g;
    MCMF(int _n) {
        n = _n + 1;
        g.resize(n);
    }
    void addEdge(int u, int v, int cap, int cost) {
        Edge e1 = {v, cost, cap, 0, (int) g[v].size()};
        Edge e2 = {u, -cost, 0, 0, (int) g[u].size()};
        g[u].push_back(e1);
        g[v].push_back(e2);
    }
    pair<int, int> minCostMaxFlow(int s, int t, int k) {
        int flow = 0;
        int cost = 0;
        vector<int> state(n), from(n), from_edge(n);
        vector<int> d(n);
        deque<int> q;
        while (flow < k) {
            for (int i = 0; i < n; i++)
                state[i] = 2, d[i] = inf, from[i] = -1;
            state[s] = 1;
            q.clear();
            q.push_back(s);
            d[s] = 0;
            while (!q.empty()) {
                int v = q.front();
                q.pop_front();
                state[v] = 0;
                for (int i = 0; i < (int) g[v].size(); i++) {
                    Edge e = g[v][i];
                    if (e.flow >= e.cap || (d[e.to] <= d[v] + e.cost))
                        continue;
                    int to = e.to;
                    d[to] = d[v] + e.cost;
                    from[to] = v;
```



```

        from_edge[to] = i;
        if (state[to] == 1) continue;
        if (!state[to] || (!q.empty() && d[q.front()] > d[to]))
            q.push_front(to);
        else q.push_back(to);
        state[to] = 1;
    }
}
if (d[t] == inf) break;
int it = t, addflow = inf;
while (it != s) {
    addflow = min(addflow,
        g[from[it]][from_edge[it]].cap
        - g[from[it]][from_edge[it]].flow);
    it = from[it];
}
if (addflow)
    addflow = min(k - flow, addflow);
it = t;
while (it != s) {
    g[from[it]][from_edge[it]].flow += addflow;
    g[it][g[from[it]][from_edge[it]].backEdge].flow -= addflow;
    cost += g[from[it]][from_edge[it]].cost * addflow;
    it = from[it];
}
flow += addflow;
}
if (flow < k)
    return {-1, -1};
return {cost, flow};
}
};

```

SCC

```

int const N = 5e5 + 5;
vector<int> adj[N], rev[N], scc[N], comp;
int id[N], n, m;

void dfs(int u, vector<int> g[], bool vis[]) {
    vis[u] = 1;
    for (auto &v: g[u]) {
        if (vis[v]) continue;
        dfs(v, g, vis);
    }
}

```

```

        comp.push_back(u);
    }

    void build() {
        bool vis[n + 1]{};
        for (int i = 1; i <= n; ++i) {
            if (!vis[i])
                dfs(i, adj, vis);
        }
        auto order = comp;
        reverse(order.begin(), order.end());
        memset(vis, 0, sizeof vis);
        for (auto &i: order) {
            if (!vis[i]) {
                comp.clear();
                dfs(i, rev, vis);
                for (auto &i: comp)
                    id[i] = comp.front();
            }
        }
        for (int u = 1; u <= n; ++u) {
            for (auto &v: adj[u]) {
                if (id[u] != id[v]) {
                    scc[id[u]].push_back(id[v]);
                }
            }
        }
    }
}

```

```

struct Tarjan {
    int n, compID, timer;
    stack<int> stk;
    set<int> artPoints;
    vector<bool> inStack;
    vector<int> scc, low, tin, id;
    set<pair<int, int>> bridges;
    vector<vector<int>> adj, comps;

    Tarjan(int _n, int _m) {
        n = _n;
        adj = comps = vector<vector<int>>>(_n + 1);
        inStack = vector<bool>(_n + 1);
        scc = low = tin = id = vector<int>(_n + 1);
        compID = timer = 0;
    }
}

```

```

}

pair<int, int> edge(int u, int v) {
    return {min(u, v), max(u, v)};
}

void dfs(int node, int par = -1) {
    // int children = 0; // uncomment for art points
    inStack[node] = true, tin[node] = low[node] = ++timer;
    stk.push(node);
    for (auto &child: adj[node]) {
        // if (child == par) continue; // uncomment for undirected graphs
        if (!tin[child]) {
            // children++;
            dfs(child, node);
            low[node] = min(low[node], low[child]);
            // if (low[child] > tin[node]) bridges.insert(edge(node,
child));

            // if (low[child] >= tin[node] and (~par or children > 1))
            artPoints.insert(node);
        } else if (inStack[child])
            low[node] = min(low[node], tin[child]);
    }
    // remove for artic and bridges
    if (low[node] == tin[node]) {
        compID++;
        while (true) {
            int cur = stk.top();
            stk.pop();
            inStack[cur] = false;
            scc[cur] = compID;
            comps[compID].push_back(cur);
            if (cur == node)
                break;
        }
    }
}

void bridges_tree(int u) {
    id[u] = timer;
    for (auto &v: adj[u]) {
        auto e = edge(u, v);
        if (bridges.find(e) != bridges.end() or id[v])
            continue;
        bridges_tree(v);
    }
}

```

```

}

vector<vector<int>> get_bridges_tree() {
    vector<vector<int>> g(n + 1);
    dfs(1, 1);
    timer = 1;
    for (int i = 1; i <= n; ++i) {
        if (id[i])
            continue;
        bridges_tree(i);
        timer++;
    }
    for (int i = 1; i <= n; ++i)
    {
        for (auto &j: adj[i]) {
            int u = id[i];
            int v = id[j];
            if (u != v)
                g[u].push_back(v);
        }
    }
    return g;
}

vector<vector<int>> getCondensedGraph() {
    vector<vector<int>> g(n + 1);
    for (int i = 1; i <= n; ++i) {
        if (tin[i])
            continue;
        dfs(i);
    }
    for (int i = 1; i <= n; ++i) {
        for (auto &j: adj[i]) {
            if (scc[j] != scc[i])
                g[scc[i]].push_back(scc[j]);
        }
    }
    return g;
}
};

```